

Code Migration and Software Environments Hands-On Material

Mahidhar Tatineni

San Diego Supercomputer Center

June 10, 2026

Outline

- Applications already available on Expanse (and other systems)
 - modules
 - singularity containers
- Python based applications/libraries
 - Use Galyleo to set up reusable conda environment
- R based applications
 - Building Singularity image from docker images available in public repositories
- Installing/building applications from source code
 - Example RAxML build from source that utilize dependencies already installed on Expanse

Repo for hands-on content:
<https://github.com/mahidhar/rehs.git>

Login to Expanse and clone this repository:

`git clone https://github.com/mahidhar/rehs.git`

Outline

- Applications already available on Expanse (and other systems)
 - modules
 - singularity containers
- Python based applications/libraries
 - Use Galyleo to set up reusable conda environment
- R based applications
 - Building Singularity image from docker images available in public repositories
- Installing/building applications from source code
 - Example RAxML build from source that utilize dependencies already installed on Expanse

Applications already available on Expanse

- SDSC staff have installed and made available a large suite of libraries and applications on Expanse. This includes commercially licensed software such as Q-Chem, Abaqus, and Gaussian.
- The primary approach is to make applications available using environment modules.
- Use “module spider” to find applications that are available via modules.
- Some applications/frameworks (e.g. PyTorch, TensorFlow, AlphaFold) have been made available using singularity containers.
- Today’s talk will be focused on Expanse specific options. The options are broadly implementable on the Triton Shared Computing Cluster (TSCC) as the software stack on TSCC is installed w/ a similar approach as Expanse.
- On our Kubernetes based clusters (Voyager, Nautilus/PNRP) we are mainly using a containerized approach (docker images can be used). On Nautilus/PNRP we have a gitlab + Kubernetes development environment and a Coder (<https://coder.com/>) based development environment.

Applications via modules

- Use “module spider” to find applications.

```
[mahidhar@login01 ~]$ module spider matlab

-----
matlab:
-----
Versions:
  matlab/2020b
  matlab/2022a
Other possible modules matches:
  matlab/2022b
-----

To find other possible module matches execute:

  $ module -r spider '.*matlab.*'
-----

For detailed information about a specific "matlab" package (including how to load the modules) use the module's full name. Note that names that have a trailing (E) are extensions provided by other modules.
For example:

  $ module spider matlab/2022a
-----
```

Applications via modules

- Use “module spider” to find applications.

```
[mahidhar@login01 ~]$ module spider matlab/2022b
```

```
-----  
matlab/2022b:  
-----
```

```
Versions:
```

```
matlab/2022b/lefe4oq
```

```
matlab/2022b/nmbr5dd
```

```
-----  
For detailed information about a specific "matlab/2022b" package (including how to load the modules) use the module's  
full name. Note that names that have a trailing (E) are extensions provided by other modules.
```

```
For example:
```

```
$ module spider matlab/2022b/nmbr5dd  
-----
```


Applications via modules

- Use “module spider” to find applications.

```
[mahidhar@login01 ~]$ module spider matlab/2022b/lefe4oq
```

```
matlab/2022b: matlab/2022b/lefe4oq
```

You will need to load all module(s) on any one of the lines below before the "matlab/2022b/lefe4oq" module is available to load.

```
cpu/0.17.3b
```

Help:

MATLAB (MATrix LABoratory) is a multi-paradigm numerical computing environment and fourth-generation programming language. A proprietary programming language developed by MathWorks, MATLAB allows matrix manipulations, plotting of functions and data, implementation of algorithms, creation of user interfaces, and interfacing with programs written in other languages, including C, C++, C#, Java, Fortran and Python. Note: MATLAB is licensed software. You will need to create an account on the MathWorks homepage and download MATLAB yourself. Spack will search your current directory for the download file. Alternatively, add this file to a mirror so that Spack can find it. For instructions on how to set up a mirror, see <http://spack.readthedocs.io/en/latest/mirrors.html>

Applications via modules

- “module spider” also gives the loading information
- The CPU and GPU stacks are completely independent. Do **not** mix the two as something compiled for the GPU stack will not work on the CPU nodes (which have a different architecture) and vice versa
- Usage examples are provided in:
</cm/shared/examples/sdsc>

```
[mahidhar@login02 sdsc]$ cd /cm/shared/examples/sdsc/
[mahidhar@login02 sdsc]$ ls
abaqus      bintest    dftbplus   gromacs    localscratch  namd      openacc    pyscf      raxml      tensorflow   vasp-ase
abinit      ciml       excerpt    hadoop     matlab        neuron    openmp     pytorch    si         test         visit
alphafold   classes    gamess     hpl        mpi           nsight    orca       qchem      siesta     trinity      wannier90
amber       cp2k       gaussian   lammps     mpi-openmp-hybrid nwchem    paraview   qe         spark      vasp         xpmem
```

Hands On! - Command set #1:

Working with modules

<https://github.com/mahidhar/rehs.git>

Outline

- Applications already available on Expanse (and other systems)
 - modules
 - singularity containers
- Python based applications/libraries
 - Use Galyleo to set up reusable conda environment
- R based applications
 - Building Singularity image from docker images available in public repositories
- Installing/building applications from source code
 - Example RAxML build from source that utilize dependencies already installed on Expanse

Conda installs using Galyleo (preferred)

- Galyleo is a command-line shell utility to help you launch Jupyter notebooks (<https://github.com/mkandes/galyleo>)
- galyleo supports the use of conda to configure the software environment for Jupyter sessions. Can use environment.yml to dynamically generate conda environments
- Mamba and caching flags can speed up installs and reuse by using conda-pack
- Much faster to have the conda distribution in NVMe instead of your home directory (number of files can slow I/O down) OR Lustre (number of files causes systemwide issues)

Jupyter notebooks using Galyleo

<https://github.com/mkandes/galyleo>

- Already installed on Expanse and you can setup path for Galyleo:

export PATH="/cm/shared/apps/sdsc/galyleo:\${PATH}"

- Example that sets up conda install using info in yml file:

galyleo launch --account abc123 --partition shared --cpus 4 --memory 8 --time-limit 00:30:00 --conda-yml environment.yml

```
[mahidhar@login02 example]$ more environment.yml
name: notebooks-sharing

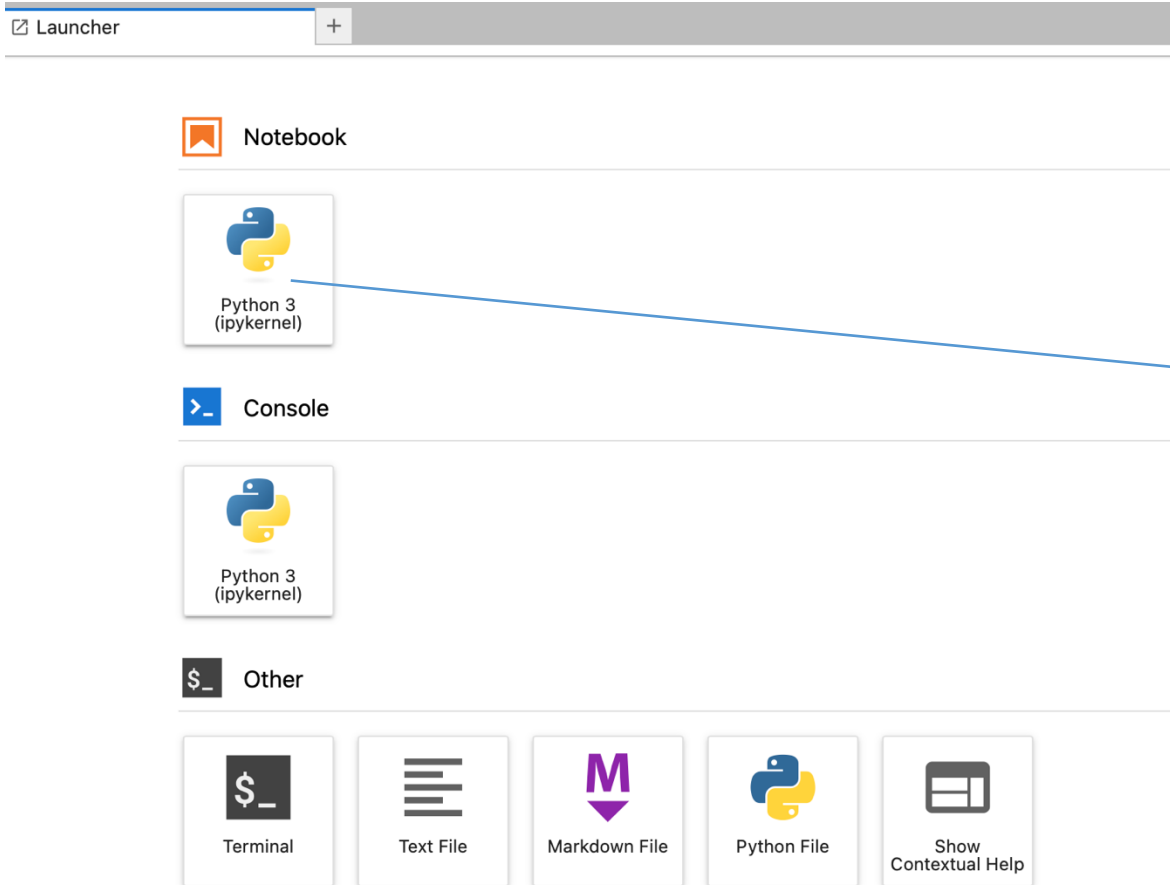
channels:
  - conda-forge

dependencies:
  - python
  - jupyterlab
  - pandas
  - matplotlib
  - seaborn
  - scikit-learn
```

```
...
Generating Jupyter launch script ...
Submitted Jupyter launch script to Slurm. Your SLURM_JOB_ID is 50579504.
Success! Token linked to jobid.
Please copy and paste the HTTPS URL provided below into your web browser.
Do not share this URL with others. It is the password to your Jupyter notebook session.
Your Jupyter notebook session will begin once compute resources are allocated to your job by the scheduler.
https://duplex-recollect-enunciate.expanse-user-content.sdsc.edu/?token=85ef[REDACTED]
```

Jupyter notebooks using Galyleo

<https://github.com/mkandes/galyleo>



Can launch a fresh Python3 notebook from the Launcher pane

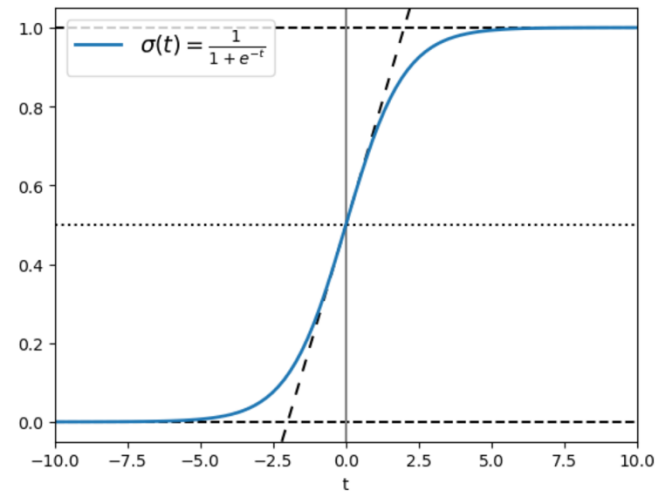
Jupyter notebooks using Galyleo

<https://github.com/mkandes/galyleo>

```
[1]: import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(-10, 10, 100)
sig = 1 / (1 + np.exp(-t))

fig, ax = plt.subplots()
ax.axhline(y=0, color="black", linestyle="--")
ax.axhline(y=0.5, color="black", linestyle=":")
ax.axhline(y=1.0, color="black", linestyle="--")
ax.axvline(color="grey")
ax.annotate((0, 0.5), slope=0.25, color="black", linestyle=(0, (5, 5)))
ax.plot(t, sig, linewidth=2, label=r"$\sigma(t) = \frac{1}{1 + e^{-t}}$")
ax.set(xlim=(-10, 10), xlabel="t")
ax.legend(fontsize=14)
plt.show()
```



[]:

Hands On! - Command set #2:

Setup a conda install in node local scratch and run Jupyterlab using Galileo

<https://github.com/mahidhar/rehs.git>

Outline

- Applications already available on Expanse (and other systems)
 - modules
 - singularity containers
- Python based applications/libraries
 - Use Galyleo to set up reusable conda environment
- R based applications
 - Building Singularity image from docker images available in public repositories
- Installing/building applications from source code
 - Example RAxML build from source that utilize dependencies already installed on Expanse

Hands On! - Command set #3

Build a Singularity image for a R application using a docker image

<https://github.com/mahidhar/rehs.git>

Outline

- Applications already available on Expanse (and other systems)
 - modules
 - singularity containers
- Python based applications/libraries
 - Use Galyleo to set up reusable conda environment
- R based applications
 - Building Singularity image from docker images available in public repositories
- Installing/building applications from source code
 - Example RAxML build from source that utilize dependencies already installed on Expanse

Installs from source, RAxML Makefile example

```
# Makefile November 2009 by Alexandros Stamatakis

CC = mpicc

CFLAGS = -D_WAYNE_MPI -D__SIM_SSE3 -O2 -D_GNU_SOURCE -msse3 -fomit-frame-pointer
        -funroll-loops -D__AVX

LIBRARIES = -lm

RM = rm -f

objs      = axml.o  optimizeModel.o multiple.o searchAlgo.o topologies.o parsePart
itions.o treeIO.o models.o bipartitionList.o rapidBootstrap.o evaluatePartialGen
ericSpecial.o evaluateGenericSpecial.o newviewGenericSpecial.o makenewzGenericSp
ecial.o  classify.o fastDNAParsimony.o fastSearch.o leaveDropping.o rmqs.o rogu
eEPA.o ancestralStates.o avxLikelihood.o  mem_alloc.o eigen.o

all : clean raxmlHPC-MPI-AVX2

GLOBAL_DEPS = axml.h globalVariables.h rmq.h rmqs.h #mem_alloc.h

raxmlHPC-MPI-AVX2 : $(objs)
        $(CC) -o raxmlHPC-MPI-AVX2 $(objs) $(LIBRARIES) $(LDFLAGS)

avxLikelihood.o : avxLikelihood.c $(GLOBAL_DEPS)
        $(CC) $(CFLAGS) -mavx2 -D_FMA -march=core-avx2 -c -o avxLikeliho
od.o avxLikelihood.c

fastDNAParsimony.o : fastDNAParsimony.c $(GLOBAL_DEPS)
        $(CC) $(CFLAGS) -mavx -c -o fastDNAParsimony.o fastDNAParsimo
```

Installs from source

- All GPU compiles **must** be done on a GPU node. Also, don't mix installs from GPU and CPU stack
- Compiles done on the login node will fail on a GPU node
- CPU codes can be compiled on the login node as the processor matches
- Note on BLAS/LAPACK/SCALAPACK: there are several options
 - OpenBLAS,
 - Netlib Scalapack
 - MKL
 - AOCL
- Libraries will be in non-standard locations. So, make sure to use configure/cmake/makefile options to point build scripts to the right locations
- Intel compilers work fine on AMD nodes. Don't use "-xHOST", switch to "-march=core-avx2"

Hands On! - Command set #4

*Downloading and building RAxML
code using a Makefile*

<https://github.com/mahidhar/rehs.git>